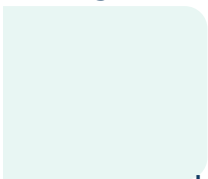
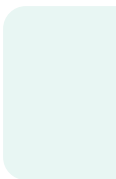


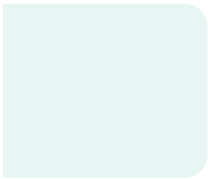
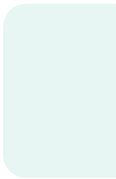


DOCUMENTO DE RELEASE

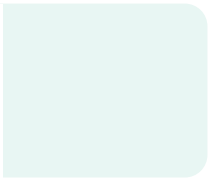
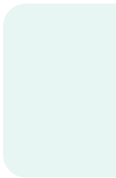
SEAL · Deployment & Quality Assurance con IA



Hugo Mauricio Romero Rodríguez
Miguel Ángel Leal Pérez
Miguel Ángel Durazno Martínez
Omar Hernández Cervantes



DS01SV-25



Héctor Saldaña Benítez
Desarrollo Web Integral

11 de agosto de 2026
San Juan del Río, Qro.

Índice

Índice	2
1. Resumen del release	3
2. Fase 1: despliegue e infraestructura	4
3. Verificación de HTTPS y contenedores	5
4. Fase 2: plan de pruebas asistido por IA	6
5. Resultados de pruebas unitarias y caja blanca	7
6. Resultados de integración con Playwright	8
7. Prueba de rendimiento con k6	9
8. Auditoría Lighthouse y WCAG 2.1	10
9. Fase 3: política de liberación	11
10. Protocolo de rollback	12
11. Normativa y privacidad	13
Anexo A. Código fuente de las pruebas	14

1. Resumen del release

SEAL es una aplicación web para la elaboración, administración y firma de contratos. La versión evaluada se publicó en una máquina virtual de Azure utilizando contenedores Docker. Nginx recibe las conexiones públicas, aplica HTTPS y dirige cada solicitud al frontend Next.js o al backend Express.

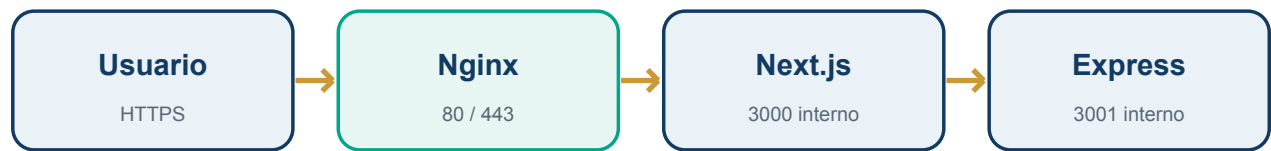
El plan de calidad se construyó con apoyo de IA y después se adaptó al funcionamiento real de SEAL. Se ejecutaron cinco grupos de pruebas: unitarias, caja blanca, integración, rendimiento y usabilidad/accesibilidad. Todas las pruebas funcionales aprobaron y los puntajes solicitados por la práctica fueron superados.

Sitio: <https://seal.westus2.cloudapp.azure.com>
Repositorio: <https://github.com/Buchito10/SEAL>

Criterio	Evidencia	Estado
Despliegue en nube	Azure VM, Docker Compose, Next.js y Express activos.	CUMPLE
HTTPS/TLS	Nginx, redirección HTTP→HTTPS y certificado público de Let's Encrypt.	CUMPLE
Pruebas con IA	17 Jest, 4 Playwright, k6 y Lighthouse ejecutados.	CUMPLE
Gobernanza	SemVer, release, rollback y privacidad documentados.	CUMPLE

2. Fase 1: despliegue e infraestructura

Arquitectura utilizada



Red Docker privada: los puertos internos no se publican en Internet

Docker Compose mantiene los componentes separados. Sólo Nginx publica puertos hacia Internet. El frontend y el backend se comunican por la red privada de Docker, de modo que los puertos 3000 y 3001 no quedan expuestos al exterior.

Flujo de una solicitud

- El usuario abre el dominio utilizando HTTPS.
- Nginx presenta el certificado TLS y descifra la conexión.
- Las rutas `/api` se envían al backend Express; el resto se envía al frontend Next.js.
- El backend utiliza Firebase y el almacenamiento de la aplicación sin publicarlos directamente.

Configuración del servidor

Criterio	Evidencia	Estado
Servidor	Azure for Students · VM roco-v4 · Ubuntu 22.04.	ACTIVO
Dominio	seal.westus2.cloudapp.azure.com → 4.154.29.114.	RESUELVE
Firewall	UFW y Azure NSG permiten 80/443; SSH se conserva para administración.	ACTIVO
Aislamiento	SEAL está en <code>/opt/seal</code> y no comparte contenedores con ROCO.	VERIFICADO

3. Verificación de HTTPS y contenedores

La comprobación se realizó directamente contra el sitio público. La conexión por HTTP devuelve una redirección 301 hacia HTTPS; la pantalla de inicio de sesión responde con HTTP 200 y el endpoint de salud devuelve {"ok":true}.

Certificado observado

Criterio	Evidencia	Estado
Nombre común	seal.westus2.cloudapp.azure.com	VÁLIDO
Emisor	Let's Encrypt · YE2	CONFIABLE
Inicio	11 de agosto de 2026, 15:52 UTC	VIGENTE
Vencimiento	9 de noviembre de 2026, 15:52 UTC	VIGENTE

Estado técnico

- Frontend y backend reportaron estado healthy.
- Nginx publicó únicamente 80 y 443 para SEAL.
- Certbot quedó activo para gestionar la renovación.
- La base PostgreSQL y la API de ROCO continuaron funcionando después de las pruebas.

El uso del mismo servidor no implica mezclar las aplicaciones: cada proyecto tiene directorio, red, contenedores y límites propios.

4. Fase 2: plan de pruebas asistido por IA

Se utilizó IA como apoyo para proponer casos, aserciones, datos límite y estructura de scripts. El equipo revisó cada propuesta, la adaptó a los módulos reales de SEAL, ejecutó los comandos y conservó únicamente los resultados reproducibles. No se utilizaron credenciales ni datos personales reales.

Prompt adaptado

“Actúa como QA Automation Engineer. Genera pruebas para la autenticación y el procesamiento seguro de plantillas de SEAL. Incluye Jest para casos unitarios y ramas internas, Playwright con API simulada para el login, k6 con 50 usuarios durante 30 segundos y una auditoría Lighthouse. Agrega aserciones claras y umbrales medibles.”

Criterio	Evidencia	Estado
1. Unitarias	Funciones y controladores aislados con Jest.	17/17*
2. Caja blanca	Ramas, condiciones, escape HTML y hash SHA-256.	APROBADA
3. Integración	Login, red, cookie y navegación con Playwright + mock.	4/4
4. Rendimiento	50 usuarios virtuales, 30 s, p95 y errores con k6.	APROBADA
5. Usabilidad	Lighthouse: accesibilidad y mejores prácticas.	> 85

*Las 17 pruebas de Jest se distribuyen en cuatro suites que incluyen casos unitarios y de caja blanca.

5. Resultados de pruebas unitarias y caja blanca

Resultado de Jest

Criterio	Evidencia	Estado
Suites	4 ejecutadas	4 APROBADAS
Casos	17 ejecutados	17 APROBADOS
Sentencias	84.42%	CUBIERTAS
Ramas	72.58%	MEDIDAS
Funciones	87.50%	CUBIERTAS
Líneas	90.74%	CUBIERTAS

Qué se comprobó

- Extracción y validación de placeholders dentro de las plantillas.
- Sustitución de campos y comportamiento cuando falta un valor.
- Escape de contenido para impedir que HTML malicioso se inserte en el contrato.
- Generación estable de la huella SHA-256 utilizada como evidencia.
- Ramas de autenticación, validación JWT y respuestas ante sesiones inválidas.

La cobertura no significa que todo el sistema sea perfecto; indica qué porcentaje del código seleccionado fue recorrido durante esta ejecución. El resultado sirve para detectar las áreas que todavía necesitan casos adicionales.

6. Resultados de integración con Playwright

Playwright abrió el sitio público y simuló las respuestas del backend mediante interceptación de red. Así se validó el flujo completo de la pantalla sin crear usuarios de prueba ni alterar la base de datos.

Criterio	Evidencia	Estado
Login correcto	Formulario → API simulada → cookie HttpOnly → dashboard.	APROBADO
Credenciales inválidas	Muestra error y no crea sesión.	APROBADO
Cambio obligatorio	La respuesta del API dirige a cambiar-password.	APROBADO
Recuperación	El control conserva un nombre accesible.	APROBADO

Resultado: 4 de 4 escenarios aprobados en 10.0 segundos.

Comando reproducible

```
BASE_URL=https://seal.westus2.cloudapp.azure.com \  
+ npm --prefix qa run test:e2e
```


7. Prueba de rendimiento con k6

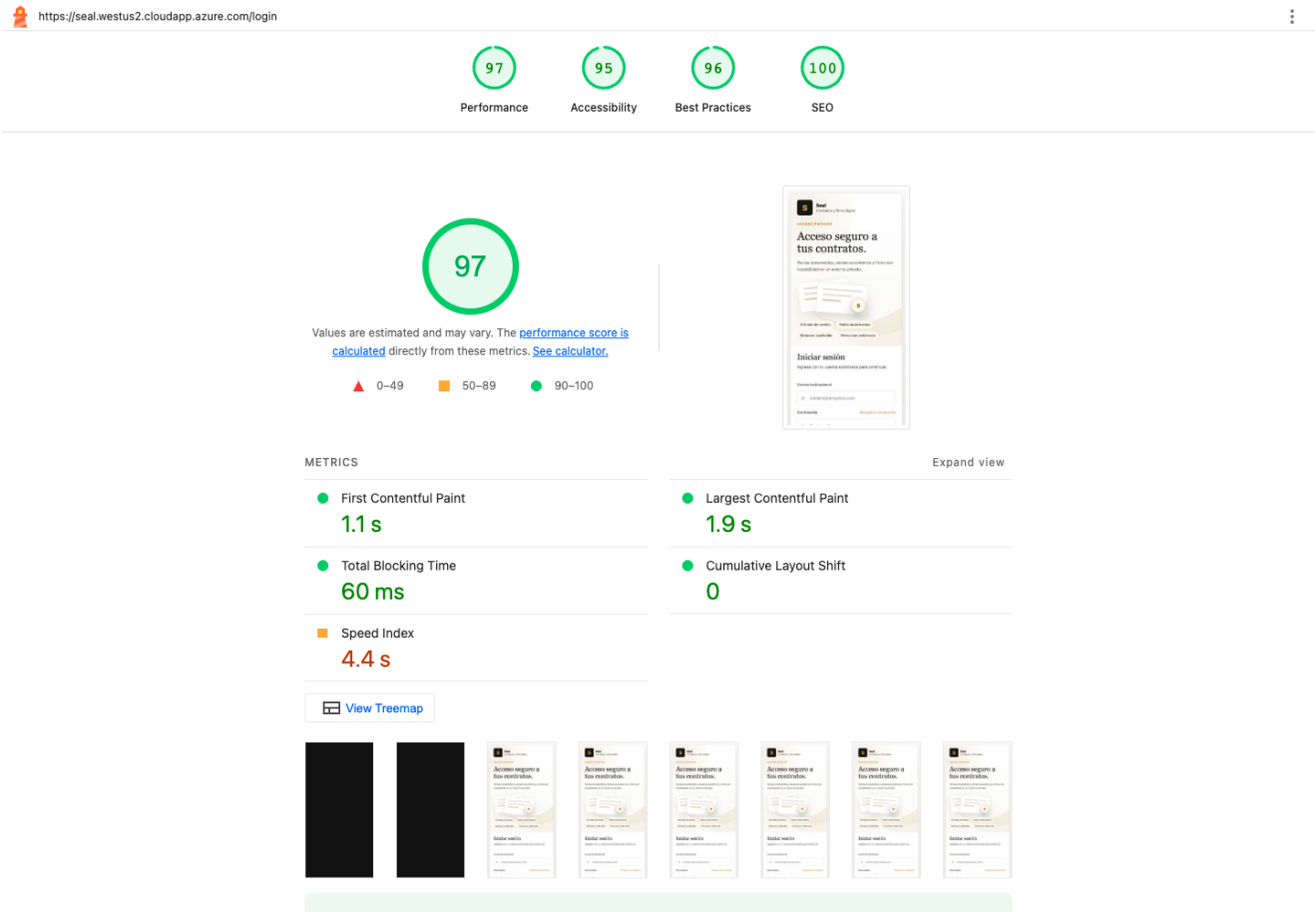
La prueba simuló 50 usuarios virtuales consultando la pantalla de login durante 30 segundos. Antes de iniciar se comprobó el endpoint de salud. Los umbrales exigieron menos de 1% de errores, percentil 95 inferior a 500 ms y más de 99% de checks aprobados.



Durante la ejecución, el frontend utilizó aproximadamente 89.88 MiB, el backend 68.61 MiB y Nginx 10.24 MiB. Los contenedores permanecieron activos y ROCO no se interrumpió.

8. Auditoría Lighthouse y WCAG 2.1

Lighthouse auditó la página pública de inicio de sesión. La práctica exige al menos 85 puntos en Accesibilidad y Mejores Prácticas; SEAL obtuvo 95 y 96 respectivamente.



Métricas principales: FCP 1.1 s, LCP 1.9 s, TBT 60 ms y CLS 0. El reporte completo HTML/JSON se conserva en `qa/reports/lighthouse`.

9. Fase 3: política de liberación

Versionamiento Git y SemVer

Las versiones usan el formato vMAJOR.MINOR.PATCH-release. MAJOR identifica cambios incompatibles, MINOR agrega funciones compatibles y PATCH corrige defectos sin romper compatibilidad. Un ejemplo para esta entrega es v1.0.0-release.

Criterios antes de publicar

- Frontend y backend compilan correctamente.
- Jest, Playwright, k6 y Lighthouse cumplen los criterios definidos.
- No hay secretos, archivos .env ni llaves Firebase dentro de Git.
- El endpoint /api/health responde y existe un punto de retorno.
- Otro integrante revisa la versión candidata.

Comandos de liberación

```
git tag -a v1.0.0-release -m "Release estable 1.0.0"
git push origin main --tags
./deploy/cloud/deploy-release.sh v1.0.0-release
```

10. Protocolo de rollback

Un rollback se utiliza cuando una versión recién publicada presenta errores críticos. El script conserva el commit anterior, detiene únicamente los servicios de SEAL, reconstruye esa versión y vuelve a comprobar el endpoint de salud.

Procedimiento

- Guardar los logs del incidente antes de cambiar la versión.
- Ejecutar `./deploy/cloud/rollback.sh` o indicar un tag conocido.
- Esperar la reconstrucción de los contenedores.
- Comprobar HTTPS, `/login`, `/api/health` y el estado `healthy`.
- Registrar versión, hora, síntomas, responsable y causa encontrada.

```
cd /opt/seal
docker compose logs --since=30m nginx frontend backend > incidente-release.log
./deploy/cloud/rollback.sh v0.9.0-release
```

Durante la exposición se explica el comando, pero no es necesario ejecutarlo sobre la versión estable. Esto evita provocar una interrupción sólo para demostrar el procedimiento.

11. Normativa y privacidad

SEAL puede tratar nombres, correos, contratos, firmas y evidencias. La directiva adopta principios de la LFPDPPP mexicana y, cuando resulte aplicable, del GDPR.

Criterio	Evidencia	Estado
Finalidad y consentimiento	Informar para qué se usan los datos antes de tratarlos.	DEFINIDO
Minimización	Solicitar únicamente datos necesarios para el contrato.	DEFINIDO
Seguridad	HTTPS, control por roles, sesiones protegidas y bitácora.	IMPLEMENTA DO
Derechos	Acceso, rectificación, cancelación y oposición; derechos GDPR aplicables.	DOCUMENTA DO
Conservación	Eliminar o anonimizar cuando termine la finalidad y el plazo legal.	DEFINIDO

Avisos dentro de la interfaz

- /politica-privacidad: aviso general, finalidades, transferencias y contacto.
- /proteccion-datos: derechos de las personas y medidas de protección.
- Las evidencias académicas deben usar cuentas y contratos ficticios.

Las contraseñas, tokens, archivos .env y llaves privadas nunca se muestran en el video ni se suben al repositorio público.

Anexo A. Código fuente de las pruebas

A continuación se incluye el código fuente principal generado con asistencia de IA y adaptado al proyecto. Los archivos completos y ejecutables también forman parte del repositorio.

Código fuente — Prueba unitaria con Jest

Archivo: Back/tests/unit/placeholders.unit.test.js

```
1  const {
2    extractPlaceholders,
3    validatePlaceholdersOrThrow,
4  } = require("../src/utis/placeholders");
5
6  describe("Lógica de plantillas de contratos", () => {
7    test("extrae placeholders sin repetirlos", () => {
8      const html = `
9        <p>{{ employee.name }}</p>
10       <p>{{ company.salary }}</p>
11       <p>{{ employee.name }}</p>
12     `;
13
14     expect(extractPlaceholders(html)).toEqual([
15       "employee.name",
16       "company.salary",
17     ]);
18   });
19
20   test("acepta únicamente placeholders del catálogo de SEAL", () => {
21     const result = validatePlaceholdersOrThrow(
22       "Contrato para {{ employee.name }} con puesto {{ company.position }}"
23     );
24
25     expect(result).toEqual({
26       used_placeholders: ["employee.name", "company.position"],
27     });
28   });
29
30   test("rechaza placeholders desconocidos", () => {
31     expect(() =>
32       validatePlaceholdersOrThrow("{{ employee.password }}")
33     ).toThrow("Invalid placeholder(s)");
34
35     try {
36       validatePlaceholdersOrThrow("{{ employee.password }}");
37     } catch (error) {
38       expect(error.code).toBe(400);
39       expect(error.details.invalid_placeholders).toEqual([
40         "employee.password",
41       ]);
42     }
43   });
44 });
```

Código fuente — Prueba de caja blanca

Archivo: Back/tests/whitebox/template-render.whitebox.test.js

```

1  const { escapeHtml, sha256Hex } = require("../src/utils/html");
2  const { renderTemplateHtml } = require("../src/utils/placeholderRender");
3  const {
4    validatePlaceholdersOrThrow,
5  } = require("../src/utils/placeholders");
6
7  describe("Caja blanca del renderizado de contratos", () => {
8    test.each([
9      [null, ""],
10     [undefined, ""],
11     ["<script>", "&lt;script&gt;"],
12     ["A&B "Legal"', "A&amp;B &quot;Legal&quot;"],
13     ["O' Connor", "O&#39;Connor"],
14   ])(`recorre las ramas de escapeHtml para %p`, (input, expected) => {
15     expect(escapeHtml(input)).toBe(expected);
16   });
17
18   test("reemplaza tokens heredados y data-ph y elimina valores ausentes", () => {
19     const template = [
20       '<span data-ph="employee.name">Nombre</span>',
21       "{{ company.position }}",
22       "{{ employee.phone }}",
23     ].join(" | ");
24
25     const result = renderTemplateHtml(template, {
26       "employee.name": "Ana <Admin>",
27       "company.position": "QA & Seguridad",
28     });
29
30     expect(result).toBe(
31       "Ana &lt;Admin&gt; | QA &amp; Seguridad | "
32     );
33   });
34
35   test("rechaza la rama data-ph antes de revisar el catálogo", () => {
36     try {
37       validatePlaceholdersOrThrow('<span data-ph="employee.name">x</span>');
38       throw new Error("Se esperaba que la validación rechazara data-ph");
39     } catch (error) {
40       expect(error).toMatchObject({
41         code: 400,
42         details: { reason: "DISALLOWED_DATA_PH" },
43       });
44     }
45   });
46
47   test("genera un hash SHA-256 estable para la evidencia documental", () => {
48     expect(sha256Hex("SEAL")).toBe(
49       "a5fd9d6c9c5442d8749ef9df86ded9c700d3688b409d78773408e95bb1d2fad4"
50     );
51   });
52 });

```

Código fuente — Prueba de integración con Playwright

Archivo: qa/e2e/login.integration.spec.js

```

1  const { test, expect } = require("@playwright/test");
2
3  test.describe("Integración del inicio de sesión", () => {
4    test("envía el formulario a la API, guarda la sesión y abre el panel", async ({ page }) => {
5      let loginPayload;
6
7      await page.route("**/api/auth/login", async (route) => {
8        loginPayload = route.request().postDataJSON();
9        await route.fulfill({
10          status: 200,
11          contentType: "application/json",
12          headers: {
13            "set-cookie": "seal_session=cookie-qa-firmada; Path=/; HttpOnly; SameSite=Lax",
14          },
15          body: JSON.stringify({
16            ok: true,
17            data: {
18              user: {
19                id: "qa-admin",
20                name: "Administración QA",
21                email: "qa@seal.test",
22                role: "ADMIN",
23                status: "ACTIVE",
24                must_change_password: false,
25              },
26            },
27          }));
28      });
29    });
30
31    await page.route("**/api/auth/session", (route) =>
32      route.fulfill({
33        status: 200,
34        contentType: "application/json",
35        body: JSON.stringify({
36          ok: true,
37          data: {
38            user: {
39              id: "qa-admin",
40              name: "Administración QA",
41              email: "qa@seal.test",
42              role: "ADMIN",
43              status: "ACTIVE",
44              must_change_password: false,
45            },
46          },
47        }));
48    });
49  });
50
51  await page.route("**/api/admin/**", (route) =>
52    route.fulfill({
53      status: 200,
54      contentType: "application/json",
55      body: JSON.stringify({ ok: true, data: [] }),
56    })
57  );
58
59  await page.goto("/login");
60  const loginForm = page.locator("form.login__card--form");
61  await loginForm.getByPlaceholder("nombre@empresa.com").fill("qa@seal.test");
62  await loginForm.getByPlaceholder("Contraseña").fill("PruebaSegura123!");
63  await loginForm.getByRole("button", { name: "Entrar" }).click();

```


Código fuente — Prueba de integración con Playwright (continuación)

Archivo: qa/e2e/login.integration.spec.js

```

64
65     await expect(page).toHaveURL(/\/$/);
66     await expect(page.getByRole("heading", { name: "Dashboard" })).toBeVisible();
67
68     expect(loginPayload).toEqual({
69       email: "qa@seal.test",
70       password: "PruebaSegura123!",
71     });
72
73     const session = await page.evaluate(() => ({
74       token: localStorage.getItem("seal_token"),
75       user: JSON.parse(localStorage.getItem("seal_user") || "null"),
76     }));
77
78     expect(session.token).toBeNull();
79     expect(session.user.role).toBe("ADMIN");
80
81     const cookies = await page.context().cookies();
82     const authCookie = cookies.find((cookie) => cookie.name === "seal_session");
83     expect(authCookie).toBeTruthy();
84     expect(authCookie.httpOnly).toBe(true);
85   });
86
87   test("muestra el error entregado por la API sin crear una sesión", async ({ page }) => {
88     await page.route("**/api/auth/login", (route) =>
89       route.fulfill({
90         status: 401,
91         contentType: "application/json",
92         body: JSON.stringify({
93           ok: false,
94           message: "Credenciales inválidas",
95         }),
96       })
97     );
98
99     await page.goto("/login");
100     const loginForm = page.locator("form.login__card--form");
101     await loginForm.getByPlaceholder("nombre@empresa.com").fill("error@seal.test");
102     await loginForm.getByPlaceholder("Contraseña").fill("Incorrecta123!");
103     await loginForm.getByRole("button", { name: "Entrar" }).click();
104
105     await expect(page.getByText("Credenciales inválidas")).toBeVisible();
106     await expect(page).toHaveURL(/\/login$/);
107     expect(await page.evaluate(() => localStorage.getItem("seal_token"))).toBeNull();
108   });
109
110   test("obliga a cambiar la contraseña antes de abrir el panel", async ({ page }) => {
111     await page.route("**/api/auth/login", (route) =>
112       route.fulfill({
113         status: 200,
114         contentType: "application/json",
115         headers: { "set-cookie": "seal_session=cambio-obligatorio; Path=/; HttpOnly; SameSite=Lax" },
116         body: JSON.stringify({
117           ok: true,
118           data: {
119             user: {
120               id: "qa-client",
121               name: "Cliente QA",
122               email: "cliente@seal.test",
123               role: "CLIENT",
124               status: "ACTIVE",
125               must_change_password: true,
126             },

```

Código fuente — Prueba de integración con Playwright (continuación)

Archivo: qa/e2e/login.integration.spec.js

```
127         },
128       },
129     })
130   );
131
132   await page.route("**/api/auth/session", (route) =>
133     route.fulfill({
134       status: 200,
135       contentType: "application/json",
136       body: JSON.stringify({
137         ok: true,
138         data: {
139           user: {
140             id: "qa-client",
141             name: "Cliente QA",
142             email: "cliente@seal.test",
143             role: "CLIENT",
144             status: "ACTIVE",
145             must_change_password: true,
146           },
147         },
148       }),
149     })
150   );
151
152   await page.goto("/login");
153   await page.getByPlaceholder("nombre@empresa.com").fill("cliente@seal.test");
154   await page.getByPlaceholder("Contraseña").fill("Temporal123!");
155   await page.getByRole("button", { name: "Entrar" }).click();
156
157   await expect(page).toHaveURL(/\/cambiar-password$/);
158   await expect(page.getByRole("heading", { name: "Cambia tu contraseña" })).toBeVisible();
159 });
160
161 test("el botón de recuperación tiene un nombre accesible independiente", async ({ page }) => {
162   await page.goto("/login");
163   await page.getByPlaceholder("Contraseña").fill("NoDebeAparecer123!");
164
165   const recoveryButton = page.getByRole("button", { name: "Recuperar contraseña", exact: true });
166   await expect(recoveryButton).toBeVisible();
167   await expect(recoveryButton).toHaveAccessibleName("Recuperar contraseña");
168 });
169 });
```

Código fuente — Prueba de carga con k6

Archivo: qa/k6/load-test.js

```
1 import http from "k6/http";
2 import { check, sleep } from "k6";
3
4 const BASE_URL = (__ENV.BASE_URL || "https://localhost").replace(/\/$/, "");
5
6 export const options = {
7   insecureSkipTLSVerify: BASE_URL.includes("localhost") || BASE_URL.includes("host.docker.internal"),
8   scenarios: {
9     usuarios_simultaneos: {
10       executor: "constant-vus",
11       vus: Number(__ENV.VUS || 50),
12       duration: __ENV.DURATION || "30s",
13     },
14   },
15   thresholds: {
16     http_req_failed: ["rate<0.01"],
17     http_req_duration: ["p(95)<500"],
18     checks: ["rate>0.99"],
19   },
20 };
21
22 export function setup() {
23   const response = http.get(`${BASE_URL}/api/health`);
24   check(response, {
25     "API disponible antes de la carga": (res) => res.status === 200,
26   });
27 }
28
29 export default function () {
30   const response = http.get(`${BASE_URL}/login`, {
31     tags: { endpoint: "login" },
32   });
33
34   check(response, {
35     "login responde 200": (res) => res.status === 200,
36     "respuesta contiene HTML": (res) =>
37       String(res.headers["Content-Type"] || "").includes("text/html"),
38     "respuesta menor a 500 ms": (res) => res.timings.duration < 500,
39   });
40
41   sleep(1);
42 }
```